

Diffusion models and flow matching

Shihua Zhang

December 26, 2024

- 1 Brief introduction to SDEs
 - From ODE to SDE
 - Score matching
- 2 From SDEs to diffusion models
- 3 Accelerated sampling
- 4 Flow matching and its connections with other generative models
- 5 Flow matching for learning on manifolds

Brief introduction to SDE

Purpose: introduce the stochastic differential equation (SDE) and its relationship with PDE and ODE [Oksendal, 2013].

- Brownian motion and SDEs
- Fokker-Plank equation and three corollaries:
 - Reverse-time SDE
 - Probability flow ODE
 - Stationary distribution of SDE

Brownian motion

Brownian motion

Brownian motion is the random motion of particles suspended in a liquid.

Definition: Suppose that the position of a particle is B_t at time t , then its position $B_{t+\Delta t}$ at time $t + \Delta t$ is $B_t + \alpha N(0, \Delta t)$. If $\alpha = 1$, then the random motion is a standard Brownian motion.

Brownian motion

Brownian motion

Brownian motion is the random motion of particles suspended in a liquid.

Definition: Suppose that the position of a particle is B_t at time t , then its position $B_{t+\Delta t}$ at time $t + \Delta t$ is $B_t + \alpha N(0, \Delta t)$. If $\alpha = 1$, then the random motion is a **standard Brownian motion**.

A standard Brownian motion is a random process $B = \{B_t : t \in [0, +\infty)\}$ that satisfies the following **properties**:

- $B_0 = 0$
- **Stationary increments:** for $s, t \in [0, +\infty)$ with $s < t$, the distribution of $B_t - B_s$ is the same as B_{t-s} .
- **Independent increment:** for $t_1, t_2, \dots, t_n \in [0, +\infty)$, with $t_1 < t_2 < \dots < t_n$, the random variable $B_{t_1}, B_{t_2} - B_{t_1}, \dots, B_{t_n} - B_{t_{n-1}}$ are independent.
- $B_t \sim N(0, t)$ for each $t \in [0, +\infty)$.
- With probability 1, B_t is continuous.

From ODE to SDE

Let's start from an ODE:

$$\frac{dX_t}{dt} = f(X_t, t)$$

Sometimes we need a “noise” term to better describe the system:

$$\frac{dX_t}{dt} = f(X_t, t) + \text{“noise”}$$

to be more specific:

$$\frac{dX_t}{dt} = f(X_t, t) + g(X_t, t) \cdot \text{“noise”}$$

We usually need the noise to be **stationary**, **independent** and **zero mean**.

From ODE to SDE

Let's start from an ODE:

$$\frac{dX_t}{dt} = f(X_t, t)$$

Sometimes we need a “noise” term to better describe the system:

$$\frac{dX_t}{dt} = f(X_t, t) + \text{“noise”}$$

to be more specific:

$$\frac{dX_t}{dt} = f(X_t, t) + g(X_t, t) \cdot \text{“noise”}$$

We usually need the noise to be **stationary**, **independent** and **zero mean**. Thus, the tiny increment of a standard Brownian motion is a satisfying candidate for noise term. A **SDE** is defined as follows:

$$dX_t = \underbrace{f(X_t, t) dt}_{\text{drift term}} + \underbrace{g(X_t, t) dB_t}_{\text{diffusion term}}$$

Examples: Diffusion process

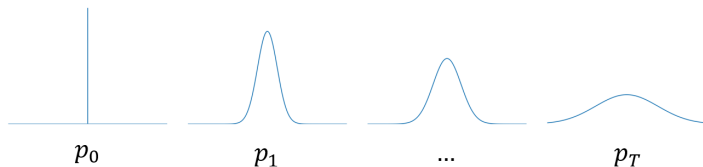
Consider a SDE with no drift term and the diffusion term is equal to 1:

$$dX_t = 0 dt + dB_t, X_0 = 0$$

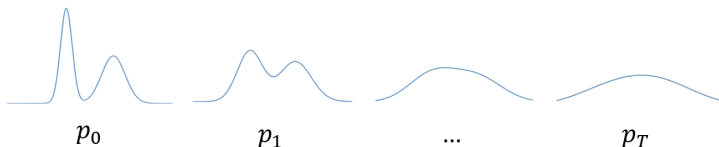
At time $t = 0$, the particle is deterministically at position 0.

For $t > 0$, the particle follows a Gaussian dist. with mean 0 and variance t .

Let p_t be the marginal distribution of X_t , we have:



Generally, if $p_0 \neq \delta$, we have:



Solution of SDE

For simplicity, ignore definitions of Itô calculus, strong and weak solutions.

Solution of SDE

The solution of a SDE is a random process, as noise is added at each infinitesimal time step. If certain mild conditions are satisfied, a solution to the SDE starting from a given distribution exists.

	方程	起始状态	t时刻状态	方程的解
ODE	$dx = f(x, t)dt$	$x(0) = x_0$	$x(t) = x_t$	$x \in C^1(R, R^n)$
SDE	$dX_t = f(X_t, t)dt + g(X_t, t)dB_t$	$X_0 \sim \nu_0$	$X_t \sim \nu_t$	$X = \{X_t : t \in [0, T]\}$

- An ODE is a special case of a SDE with a zero diffusion term.
- The initial state can be any general distribution rather than being limited to just a delta function.

Numerically solve SDE by Euler-Maruyama method

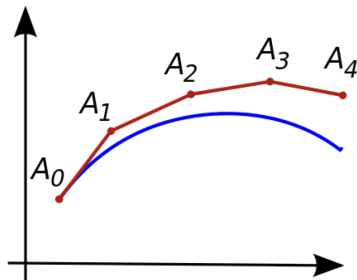
To solve an ODE of the form:

$$dX = f(X, t) dt$$

the time interval $[0, T]$ is discretized into n steps with: $t_1 = 0, \dots, t_n = T$, where the step size h is defined by $t_{i+1} = t_i + h$ ($i = 1, 2, \dots, n-1$).

ODE is numerically approximated using the **forward Euler method**:

$$x_{i+1} = x_i + f(x_i, t_i)h$$



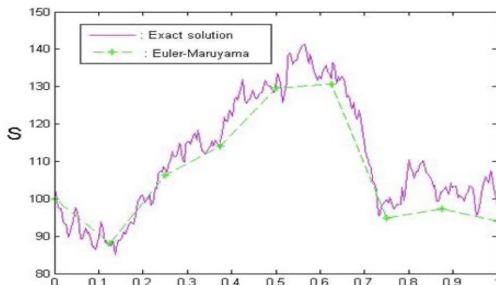
Numerically solve SDE by Euler-Maruyama method

The **Euler-Maruyama method** is the stochastic counterpart of the Euler method, applied to SDEs. Given a SDE of the form:

$$dX_t = f(X_t, t)dt + g(X_t, t)dB_t$$

we firstly discretize the time interval $[0, T]$ into n steps: $t_1 = 0, \dots, t_n = T$, where $t_{i+1} = t_i + h$. The numerical approximation of a path of the SDE is then given by the Euler-Maruyama scheme:

$$x_{i+1} = x_i + f(x_i, t_i)h + g(x_i, t_i)\Delta B_i, \Delta B_i \sim N(0, h)$$

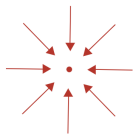


Divergence operator and Laplacian operator

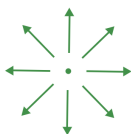
We introduce two operators to describe the change in the marginal distribution of an SDE:

- Divergence operator $\nabla \cdot$ represents the rate at which particles flow out of a given region.

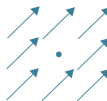
$$\nabla \cdot \vec{v} < 0$$



$$\nabla \cdot \vec{v} > 0$$



$$\nabla \cdot \vec{v} = 0$$



- Laplacian operator $\Delta = \nabla \cdot \nabla$ represents heat diffusion. The operator $-\nabla$ indicates the heat flow, while $-\nabla \cdot$ describes the rate at which heat enters the region.

$$dX_t = \underbrace{f(X_t, t)}_{\text{flow}} dt + \underbrace{g(X_t, t)}_{\text{diffusion}} dB_t$$

Fokker-Planck equation

Similar to ODEs, most SDEs do not have closed-form solutions.

However, the marginal distribution of a SDE can be represented using the Fokker-Planck equation:

Fokker-Planck equation

Given a SDE:

$$dX_t = \underbrace{f(X_t, t)}_{\text{flow}} dt + \underbrace{g(X_t, t)}_{\text{diffusion}} dB_t$$

and the initial distribution with density $p_0(x)$. The solution of the PDE yields the density function of the SDE at time t :

$$\underbrace{\frac{\partial p(x, t)}{\partial t}}_{\text{change of density}} = \underbrace{-\nabla \cdot (f(x, t)p(x, t))}_{\text{density flow in}} + \underbrace{\frac{1}{2}\Delta(g^2(x, t)p(x, t))}_{\text{density diffuse in}}$$

Reverse-time SDE

If X_t is the solution of the **SDE**:

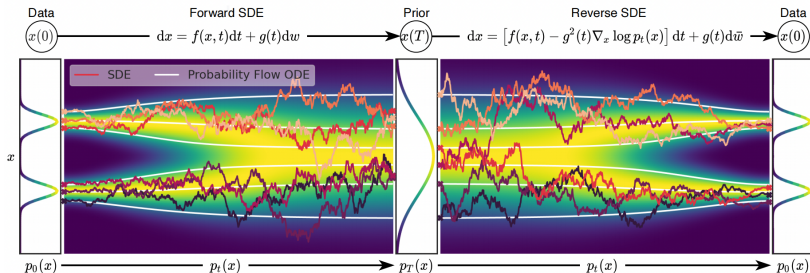
$$dX_t = f(X_t, t) dt + g(t) dB_t$$

Y_t is the solution of the **reverse-time SDE**:

$$dY_t = (f(Y_t, t) - g^2(t) \nabla_y \log p_Y(Y_t, t)) dt + g(t) dB_t$$

and $p_X(\cdot, T) = p_Y(\cdot, T)$, then:

$$p_X(\cdot, t) = p_Y(\cdot, t), \forall t \in [0, T]$$



Probability flow ODE: replace SDE by ODE

Probability flow ODE

Starting from **the same initial distribution**, the solutions of the following **SDE** and **ODE** have **the same marginal distribution** at any time t .

$$\begin{aligned}dX_t &= f(X_t, t) dt + g(t) dB_t \\dY_t &= \left(f(Y_t, t) - \frac{1}{2} g^2(t) \nabla_y \log p_Y(Y_t, t) \right) dt\end{aligned}$$

Utilizing this property, the SDE system can be replaced with an ODE system that shares the same marginal distribution.

Key: This substitution enables de-randomization, thereby accelerating the solution process in diffusion models.

Stationary distribution of SDE

A stationary distribution is the ‘fixed point’ of a SDE and often the limiting distribution.

Stationary distribution of SDE

Consider the diffusion process given by the SDE

$$dX_t = f(X_t) dt + g(X_t) dB_t$$

where $X_0 \sim \nu$ with density $p(x)$. The distribution ν is called stationary distribution of diffusion process X_t if for any t the distribution of X_t is the same as ν .

Stationary distribution of SDE

A stationary distribution is the 'fixed point' of a SDE and often the limiting distribution.

Stationary distribution of SDE

Consider the diffusion process given by the SDE

$$dX_t = f(X_t) dt + g(X_t) dB_t$$

where $X_0 \sim \nu$ with density $p(x)$. The distribution ν is called stationary distribution of diffusion process X_t if for any t the distribution of X_t is the same as ν .

The density function $p(x)$ satisfies the following equation:

$$-\nabla \cdot (f(x)p(x)) + \frac{1}{2}\Delta(g^2(x)p(x)) = 0$$

Score-based generative models

Langevin dynamics

Consider the following diffusion process given by SDE:

$$dX_t = -\nabla V(X_t) dt + \sqrt{2} dB_t$$

The density function of stationary distribution is proportional to $e^{-V(x)}$.

Let $V(x)$ be $-\log p(x)$, the stationary distribution of Langevin dynamics will be $p(x)$ (under some conditions, for example support of $p(x)$ is connected).

We call the $\nabla \log p(x)$ **score function** (different from statistics).

This property shows that given $\nabla \log p(x)$, we can sample from $p(x)$ by solving the SDE.

As a result, learning the real distribution is equivalent to learning the score function of the real distribution!

Score matching

To learn the score function of real distribution, we aim to use a neural network to approximate $\nabla \log p_{\text{real}}(x)$ by optimizing the following explicit score matching (ESM) loss over θ :

$$J_{ESMp_{\text{real}}}(\theta) = \mathbb{E}_{p_{\text{real}}(x)} \left[\frac{1}{2} \|\phi_{\theta}(x) - \nabla \log p_{\text{real}}(x)\|^2 \right]$$

We don't know $p_{\text{real}}(x)$ so we can't optimize J_{ESM} directly.

We use $p_{\text{empirical}}$ or $p_{\text{empirical}} + \text{noise}$ to substitute p_{real} in training process.

Adding noise to empirical distribution

Define the joint density

$$p_{\sigma}(x, \tilde{x}) = p_{\sigma}(\tilde{x}|x)p_{\text{empirical}}(x)$$

where $p_{\sigma}(\tilde{x}|x) \sim N(x, \sigma^2)$. Marginal distribution $p_{\sigma}(\tilde{x})$ is the noised empirical distribution.

Score matching

Define the denoising score matching (DSM) loss:

$$\begin{aligned} J_{DSMp_\sigma}(\theta) &= \mathbb{E}_{p_\sigma(\tilde{x}, x)} \left[\frac{1}{2} \left\| \phi_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}|x) \right\|^2 \right] \\ &= \mathbb{E}_{p_{\text{empirical}}(x)} \mathbb{E}_{p_\sigma(\tilde{x}|x)} \left[\frac{1}{2} \left\| \phi_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}|x) \right\|^2 \right] \end{aligned}$$

ESM and DSM [Vincent, 2011]

It is proved that:

$$J_{DSMp_\sigma}(\theta) = J_{ESMp_\sigma}(\theta) + C_1$$

which means these two targets are equivalent in terms of optimization.

Brief summary

- Brief introduction to SDE
 - Brownian motion and stochastic differential equations
 - Fokker-Plank equation
 - Reverse-time SDE
 - Probability flow ODE
 - Stationary distribution of SDE
- Langevin dynamics and score matching

- 1 Brief introduction to SDEs
- 2 From SDEs to diffusion models**
 - Denoising diffusion probabilistic models (DDPM): Discretized SDE
 - Recover the Loss function of DDPM by ELBO
 - Other diffusion models
- 3 Accelerated sampling
- 4 Flow matching and its connections with other generative models
- 5 Flow matching for learning on manifolds

SDE for generation

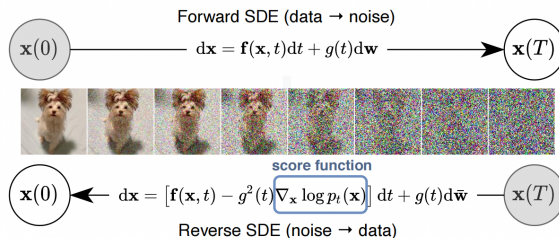


Figure: SDE and reverse-time SDE

- Generate new samples by solving the reverse-time SDE.
- Let the forward SDE $dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$.
- **How to learn the score function?**

Answer: Achieved by the score matching technique.

DDPM: Discretized SDE [Song et al., 2020]

Discretize the forward SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

We have:

$$x_i = \sqrt{1 - \beta_i}x_{i-1} + \sqrt{\beta_i}z_{i-1}, i = 1, \dots, N$$

where $z_{i-1} \sim \mathcal{N}(0, 1)$, $N\beta_i = \beta(\frac{i}{N})$. As $N \rightarrow \infty$, the upper Markov chain goes to SDE.

DDPM: Discretized SDE [Song et al., 2020]

Discretize the forward SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

We have:

$$x_i = \sqrt{1 - \beta_i}x_{i-1} + \sqrt{\beta_i}z_{i-1}, i = 1, \dots, N$$

where $z_{i-1} \sim \mathcal{N}(0, 1)$, $N\beta_i = \beta(\frac{i}{N})$. As $N \rightarrow \infty$, the upper Markov chain goes to SDE. Similarly, discretize the backward SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t - \beta(t)S_\theta(X_t, t) dt + \sqrt{\beta(t)} dB_t$$

we have:

$$x_i = \frac{1}{\sqrt{\alpha_{i-1}}} \left(x_{i-1} - \frac{\beta_{i-1}}{\sqrt{1 - \bar{\alpha}_{i-1}}} \epsilon_\theta(x_{i-1}, t) \right) + \sqrt{\beta_{i-1}} z_{i-1}, i = 1, \dots, N$$

where the $\alpha_i = 1 - \beta_i$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, $\frac{\beta_{i-1}}{\sqrt{1 - \bar{\alpha}_{i-1}}} \epsilon_\theta(x_{i-1}, t) = S_\theta(X_t, t)$.

DDPM: training and sampling

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
 - 6: **until** converged
-

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

Framework of DDPM: a hierarchical VAE

Diffusion model [Ho et al., 2020] can be regarded as a hierarchical VAE model with three key ideas [Luo, 2022]:

- The latent dimension is exactly equal to the data dimension.
- The structure of the latent encoder at each timestep is not learned; it is a Gaussian distribution centered around the out put of the previous timestep $q(x_t|x_{t-1}) \sim \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)\mathbf{I})$.
- The Gaussian parameters of the latent encoders vary over time so that the distribution of the latent at final timestep T is a standard Gaussian.

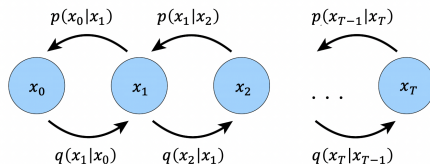


Figure: hierarchical VAE

Recover loss function of DDPM by ELBO

The target function of the diffusion model can be explained by the extended ELBO:

$$\begin{aligned}\log p_{\theta}(x_0) &= \log \int p_{\theta}(x_0, x_{1:T}) dx_{1:T} \\ &= \log \int \frac{p_{\theta}(x_0, x_{1:T}) q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} dx_{1:T} \\ &= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_{\theta}(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right] \\ &\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_{\theta}(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right] (\text{ELBO})\end{aligned}$$

By calculation, we have

$$\begin{aligned}
 \text{ELBO} &= \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right] \\
 &= \underbrace{\mathbb{E}_{q(x_1|x_0)} \log p_\theta(x_0|x_1)}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}(q(x_T|x_0) \| p(x_T))}_{\text{matching prior distribution}} \\
 &\quad - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(x_t|x_0)} \left[D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t)) \right]}_{\text{denoising matching term}}
 \end{aligned}$$

Three interpretations of diffusion model

It's clear that $q(x_{t-1}|x_t, x_0)$ is Gaussian. Set $p_\theta(x_{t-1}|x_t) \sim \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_q(t))$, by careful calculation, KL Divergence terms in Loss of DDPM shows that:

$$\begin{aligned} D_{\text{KL}}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) &\propto \|x_\theta(x_t, t) - x_0\|_2^2 \\ &\propto \|\epsilon_\theta(x_t, t) - \epsilon_0\|_2^2 \\ &\propto \|\nabla \log q(x_t|x_0) - S_\theta(x_t, t)\|_2^2 \end{aligned}$$

Three interpretations are:

- learn to predict x_0 from noisy x_t .
- learn to predict the source noise ϵ_0 that determines x_t from x_0 .
- approximate score function $\nabla \log q(x_t)$ by neural network (by the equivalence of ESM and DSE)

Other diffusion models

Key: the DDPM is a discretized SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

Can we design new diffusion models by other SDEs?

Other diffusion models

Key: the DDPM is a discretized SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

Can we design new diffusion models by other SDEs? **Yes!**

Other diffusion models

Key: the DDPM is a discretized SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

Can we design new diffusion models by other SDEs? **Yes!**

- Different diffusion models have been proposed.
- They correspond to different discretized SDEs [Song et al., 2020].

Forward SDE	Diffusion model
$dX_t = \sqrt{\frac{d(\sigma^2(t))}{dt}} dB_t$	$x_i = x_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2} z_{i-1}$
$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$	$x_i = e^{-\frac{1}{2}\int_0^t \beta(s) ds} x_{i-1} + \sqrt{1 - e^{-\int_0^t \beta(s) ds}} z_{i-1}$
$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)(1 - e^{-2\int_0^t \beta(s) ds})} dB_t$	$x_i = e^{-\frac{1}{2}\int_0^t \beta(s) ds} x_{i-1} + (1 - e^{-\int_0^t \beta(s) ds})^{1/2} z_{i-1}$

Equivalence of ELBO in different diffusion models

ELBO of infinite latent layers

For diffusion models, $q(X_t|X_0) = N(\alpha_t X_0, \sigma_t^2) (t \in [0, 1])$.

In this diffusion process, X_0 is the signal and the scale of noise is σ_t .

Define signal noise ratio (SNR):

$$SNR(t) = \frac{\alpha_t^2}{\sigma_t^2}$$

When the time step goes to zero (meanwhile, the number of latent layers goes to infinity), the summation of KL divergence terms in ELBO goes to:

$$\frac{1}{2} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, \mathbf{I})} \int_{SNR_{min}}^{SNR_{max}} \|X_0 - \tilde{X}_\theta(X_v, v)\|_2^2 dv$$

where $\tilde{X}_\theta(X_v, v) := \hat{X}_\theta(X_t, t)$, $v := SNR(t)$.

Equivalence of ELBO in different diffusion models

ELBO of infinite latent layers [Kingma et al., 2021]

When the time step goes to zero (meanwhile, number of latent layers goes to infinity), the ELBO goes to:

$$\frac{1}{2} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, \mathbf{I})} \int_{SNR_{min}}^{SNR_{max}} \|X_0 - \tilde{X}_\theta(X_v, v)\|_2^2 dv \\ + \mathbb{E}_{q(x_1|x_0)} \log p_\theta(x_0|x_1) - D_{KL}(q(x_T|x_0) \| p(x_T))$$

which means that different diffusion models share a same target function when the diffusion model adequately approximate their SDE.

For the continuous-time (infinite-depth) setting, there is an invariance of ELBO among different diffusion models. These models share a same training dynamic.

Brief summary

- Understanding DDPM through discretized SDE
- Understanding DDPM through hierarchical VAE
- Equivalence of ELBO in different diffusion models for continuous-time setting.

- 1 Brief introduction to SDEs
- 2 From SDEs to diffusion models
- 3 Accelerated sampling**
 - Training-free sampling methods
 - Training-based sampling methods
- 4 Flow matching and its connections with other generative models
- 5 Flow matching for learning on manifolds

How to make sampling faster?

One bottleneck of diffusion models is its slowness in sampling

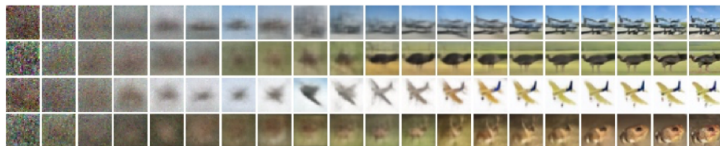
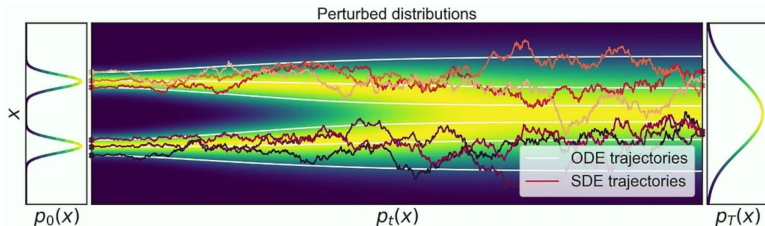


Figure: Sampling trajectory of diffusion models.

- For DDPMs, large T values, such as $T = 1000$.
- To solve the **reverse time SDE**, SDE solvers need around 1000 sequential steps to converge.

Converting the SDE to an ODE



Probability flow ODE

Starting from a same distribution, solutions of the following SDE and ODE has a same marginal distribution at any time t .

$$dX_t = f(X_t, t)dt + g(t)dB_t$$
$$dY_t = \left(f(Y_t, t) - \frac{1}{2}g^2(t)\nabla_y \log p_Y(Y_t, t) \right)dt$$

ODE solvers enable faster samplers!

Traditional Runge-Kutta Methods

Solving the reverse time probability flow ODEs:

$$\frac{dx_t}{dt} = f(t)x_t + \frac{g^2(t)}{2\sigma_t} \epsilon_{\theta}(x_t, t)$$

Given an initial value x_s at time s , the exact solution x_t at time t satisfies:

$$x_t = x_s + \underbrace{\int_s^t \left(f(\tau)x_{\tau} + \frac{g^2(\tau)}{2\sigma_{\tau}} \epsilon_{\theta}(x_{\tau}, \tau) \right) d\tau}_{\text{A "black-box"}}$$

- Traditional Runge-Kutta methods (RK45) cannot converge for < 20 steps.
- The “black-box” assumption ignores **known information** of the reverse time probability flow ODEs.

The reverse time probability flow ODE has a semi-linear structure:

$$\frac{dx_t}{dt} = f(t)x_t + \frac{g^2(t)}{2\sigma_t} \epsilon_\theta(x_t, t)$$

where $f(t)x_t$ is a **linear function**.

For linear part, we can get the exact solution x_t at t :

$$x_t = \underbrace{e^{\int_s^t f(\tau) d\tau} x_s}_{\text{Exactly computed}} + \int_s^t \left(e^{\int_\tau^t f(r) dr} \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) \right) d\tau$$

For non-linear part, simplified to an exponentially weighted integral.

$$x_t = \underbrace{\frac{\alpha_t}{\alpha_s} x_s}_{\text{Exactly computed}} - \underbrace{\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \hat{\epsilon}_\theta(\hat{x}_\lambda, \lambda) d\lambda}_{\text{Exponentially weighted integral}}$$

DPM-Solver: customized solver for diffusion ODEs

Reduce the discretization error as much as possible

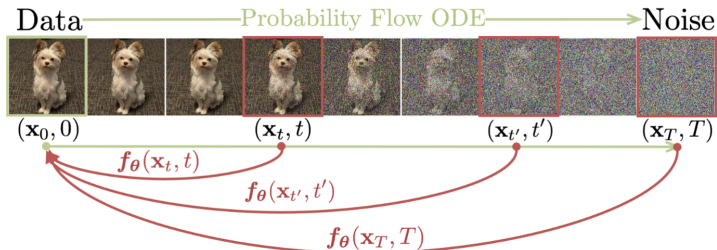
$$x_{t_{i-1} \rightarrow t_i} = \underbrace{\frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}}}_{\text{Linear term Exactly computed}} - \alpha_{t_i} \sum_{n=0}^{k-1} \underbrace{\hat{\epsilon}_{\theta}^{(n)}(\hat{x}_{\lambda_{t_{i-1}}}, \lambda_{t_{i-1}})}_{\text{Derivatives Approximated}} \underbrace{\int_{\lambda_{t_{i-1}}}^{\lambda_{t_i}} e^{-\lambda} \frac{(\lambda - \lambda_{t_{i-1}})^n}{n!} d\lambda}_{\text{Coefficients Exactly computed}} + \underbrace{\mathcal{O}(h_i^{k+1})}_{\text{High-order errors Omitted}}$$

Sampling by DPM-Solver needs only 10 ~ 20 steps without any further training.



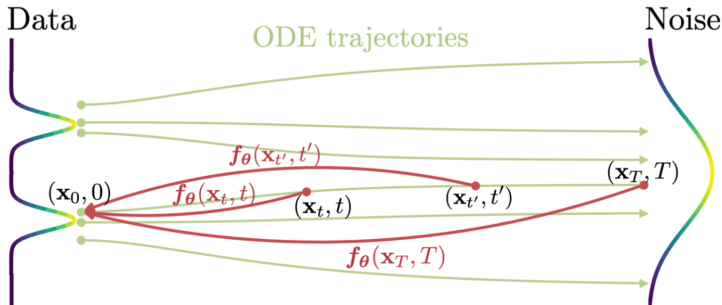
Figure: Left: DDIM. Right: DPM-Solver

Consistency models



- Consistency models [Song et al., 2023] are trained to map points on any trajectory of the probability flow ODE to the trajectory's origin in one step.
- Models of these mappings are called consistency models, as their outputs are trained to be consistent for points on the same trajectory

Enforcing self-consistency via distillation



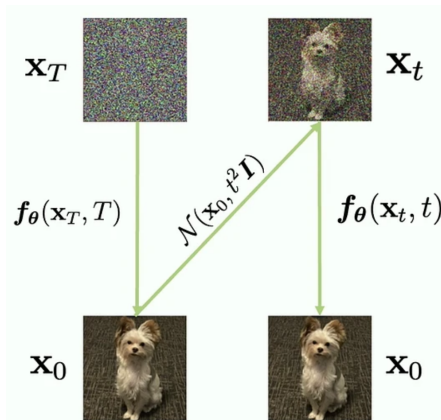
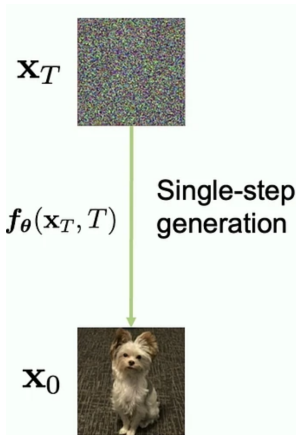
- Given a pre-trained diffusion model $s_\phi(x, t)$
- With a random time step t_{n+1} and perturbed data point $x_{t_{n+1}}$
 - Run the ODE step to move from time step t_{n+1} to time step t_n

$$\hat{\mathbf{x}}_{t_n}^\Phi := \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1})\Phi(\mathbf{x}_{t_{n+1}}, t_{n+1}; \Phi)$$

- Minimize the consistency loss

$$\min_{\theta} \lambda(t_n) \|f_\theta(x_{t_{n+1}}, t_{n+1}) - f_\theta^-(\hat{\mathbf{x}}_{t_n}^\Phi, t_n)\|_2^2$$

Sampling from consistency models



Brief summary

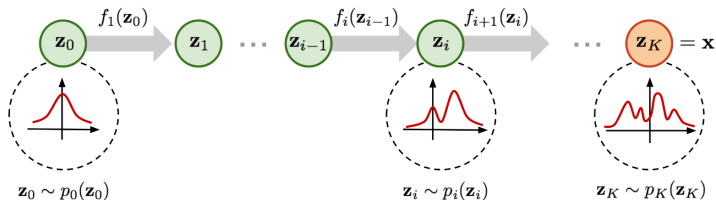
How to accelerate sampling in diffusion models?

Focusing on training-free methods like DPM-Solver, and training-based methods like consistency models.

- Training-free methods accelerate diffusion model sampling by designing **more efficient numerical solvers** for the underlying reverse time probability flow ODE.
- Training-based methods **distill** multi-step sampling into single or few steps via knowledge distillation.

- 1 Brief introduction to SDEs
- 2 From SDEs to diffusion models
- 3 Accelerated sampling
- 4 Flow matching and its connections with other generative models**
 - From Continuous Normalizing Flows to Flow Matching
 - Connections between flow matching and diffusion models
- 5 Flow matching for learning on manifolds

Normalizing flow



Normalizing: From a simple distribution to the data distribution.

Flow: A series of invertible transformations:

$$x = f_K \circ \dots \circ f_1(z_0) \triangleq f_\theta(z_0)$$

Continuous normalizing flow [Chen et al., 2018]

Generation:

$$x = f_K \circ \cdots \circ f_1(z_0) \triangleq f_\theta(z_0), \quad z_0 \sim \mathcal{N}(0, I)$$

Move from a discrete set of layers to a continuous transformation.

We get an ODE:

$$\frac{d}{dt}\phi_t(\cdot) = v_t(\phi_t(\cdot)) \quad (1)$$

$$\phi_0(\cdot) = id(\cdot) \quad (2)$$

where

- $v_t : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the time-dependent vector field.
- $\phi_t : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is called a Continuous Normalizing Flow (CNF), and it maps prior density to the ODE solution at time t .

Continuous normalizing flow

The CNF ϕ_t maps a simple prior density p_0 (e.g., Gaussian) to a more complicated one, p_1 , via the push-forward equation:

$$p_t(x) = [\phi_t]_* p_0(x) = p_0(\phi_t^{-1}(x)) \det \frac{\partial \phi_t^{-1}}{\partial x}(x) \quad (3)$$

Here $p_t : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ is a time-dependent probability density function, i.e., $\int p_t(x) dx = 1$.

- However, training CNFs (by maximum log-likelihood) can be extremely slow due to expensive numerical ODE simulations.
- Can we design a ‘simulation-free’ (i.e. not requiring any integration) training procedure for training CNF models?
 - Yes, Flow Matching.

Flow matching (FM) is a simulation-free way to train CNF models.

Given a target probability density path $p_t(x)$ and a corresponding vector field $u_t(x)$, which generates $p_t(x)$, we define the FM objective as

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, p_t(x)} \|v_{\theta, t}(x) - u_t(x)\|^2, \quad (4)$$

Note:

- The FM loss regresses the vector field u_t with a neural network $v_{\theta, t}$
- It is intractable to use in practice since
 - We have no prior knowledge for what an appropriate p_t and u_t are.
 - If we already have access to u_t , then there is no need to learn an approximation.

Conditional probability path and vector field

Given a particular data sample x_1 , we denote by $p_t(x|x_1)$ a **conditional probability path** such that:

- $p_0(x|x_1) = p_0(x)$.
- $p_1(x|x_1)$ is a distribution concentrated around $x = x_1$.

Let $u_t(\cdot|x_1) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a conditional vector field that generates the conditional probability path $p_t(\cdot|x_1)$.

We consider below conditional flow matching objective:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t,q(x_1),p_t(x|x_1)} \|v_{\theta,t}(x) - u_t(x|x_1)\|^2$$

Conditional probability path and vector field

Given a particular data sample x_1 , we denote by $p_t(x|x_1)$ a **conditional probability path** such that:

- $p_0(x|x_1) = p_0(x)$.
- $p_1(x|x_1)$ is a distribution concentrated around $x = x_1$.

Let $u_t(\cdot|x_1) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a conditional vector field that generates the conditional probability path $p_t(\cdot|x_1)$.

We consider below conditional flow matching objective:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t,q(x_1),p_t(x|x_1)} \|v_{\theta,t}(x) - u_t(x|x_1)\|^2$$

Let's recall the flow matching objective

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t,p_t(x)} \|v_{\theta,t}(x) - u_t(x)\|^2$$

Conditional flow matching objective

Connection between $\mathcal{L}_{FM}$ and $\mathcal{L}_{CFM}$

Assuming that $p_t(x) > 0$ for all $x \in \mathbb{R}^d$ and $t \in [0, 1]$, then, up to a constant independent of θ , $\mathcal{L}_{CFM}$ and $\mathcal{L}_{FM}$ are equal. Hence, $\nabla_{\theta} \mathcal{L}_{FM}(\theta) = \nabla_{\theta} \mathcal{L}_{CFM}(\theta)$.

Key: Optimizing the CFM objective is equivalent (in expectation) to optimizing the FM objective!

Conditional flow matching objective

Connection between $\mathcal{L}_{FM}$ and $\mathcal{L}_{CFM}$

Assuming that $p_t(x) > 0$ for all $x \in \mathbb{R}^d$ and $t \in [0, 1]$, then, up to a constant independent of θ , $\mathcal{L}_{CFM}$ and $\mathcal{L}_{FM}$ are equal. Hence, $\nabla_{\theta} \mathcal{L}_{FM}(\theta) = \nabla_{\theta} \mathcal{L}_{CFM}(\theta)$.

Key: Optimizing the CFM objective is equivalent (in expectation) to optimizing the FM objective!

The CFM objective allows to easily sample unbiased estimates, as long as

- efficiently sample from $p_t(x|x_1)$
- efficient compute $u_t(x|x_1)$

Design suitable conditional probability paths and vector fields.

Construction of $p_t(x|x_1)$

Consider a general family of Gaussian conditional probability paths

$$p_t(x|x_1) = \mathcal{N}(x | \mu_t(x_1), \sigma_t(x_1)^2 I)$$

for some mean $\mu_t(x_1)$ and standard deviation $\sigma_t(x_1)$.

- Set $\mu_0(x_1) = 0$ and $\sigma_0(x_1) = 1$, then $p_0(x) = \mathcal{N}(x|0, I)$
- Set $\mu_1(x_1) = x_1$ and $\sigma_1(x_1) = \sigma_{\min}$, then $p_1(x|x_1)$ is a concentrated Gaussian distribution centered at x_1

Construction of $u_t(x|x_1)$

Consider the flow:

$$\psi_t(x) = \sigma_t(x_1)x + \mu_t(x_1)$$

It provides a vector field that generates the Gaussian conditional probability path

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x)|x_1)$$

Closed form of $u_t(x|x_1)$

Let $p_t(x|x_1)$ be a Gaussian probability path, and ψ_t its corresponding flow map. Then, the unique vector field that defines ψ_t has the form:

$$u_t(x|x_1) = \frac{\sigma'_t(x_1)}{\sigma_t(x_1)}x - \mu_t(x_1) + \mu'_t(x_1). \quad (5)$$

Consequently, $u_t(x|x_1)$ generates the Gaussian path $p_t(x|x_1)$.

Example: Optimal transport conditional vector fields

Define the mean and the std to simply change linearly in time:

$$\mu_t(x) = tx_1, \text{ and } \sigma_t(x) = 1 - (1 - \sigma_{\min})t$$

The flow is:

$$\psi_t(x) = (1 - (1 - \sigma_{\min})t)x + tx_1$$

The conditional vector field is:

$$u_t(x|x_1) = \frac{x_1 - (1 - \sigma_{\min})x}{1 - (1 - \sigma_{\min})t}$$

The CFM loss takes the form:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t,q(x_1),p(x_0)} \left\| v_{\theta,t}(\psi_t(x_0)) - \left(x_1 - (1 - \sigma_{\min})x_0 \right) \right\|^2$$

Summary

- Training continuous normalizing flows are extremely slow due to expensive numerical ODE simulations.
- Flow Matching, a new simulation-free framework for training CNF models based on regressing vector fields of fixed conditional probability paths.
- Concurrent to flow matching: rectified flow [\[Liu et al., 2022\]](#) and stochastic interpolants [\[Albergo and Vanden-Eijnden, 2022\]](#).

Connections between Flow matching and Diffusion models

Connections between SDEs and ODEs

Connection between diffusion models and flow matching models

[Lipman et al., 2022]

- SDE and Probability flow ODE
- Connections between diffusion models and flow matching
 - Conditional diffusion and conditional flow
 - Benefits of flow matching models

Probability flow ODE: replace SDE by ODE

Probability flow ODE

Starting from the same initial distribution, the solutions of the following SDE and ODE have the same marginal distribution at any time t .

$$\begin{aligned}dX_t &= f(X_t, t) dt + g(t) dB_t \\dY_t &= \left(f(Y_t, t) - \frac{1}{2} g^2(t) \nabla_x \log p_Y(Y_t, t) \right) dt\end{aligned}$$

Utilizing this property, the SDE system can be replaced with an ODE system that shares the same marginal distribution.

This substitution enables de-randomization, thereby accelerating the solution process in diffusion models.

Flow matching: Replace SDE by ODE

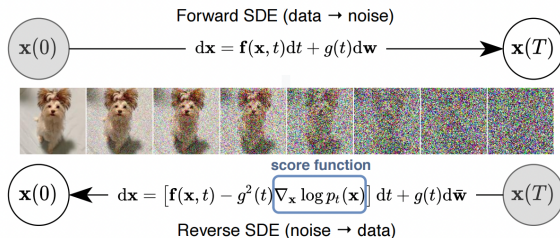


Figure: SDE and reverse-time SDE

DM fundamentally design a forward SDE to introduce noise and learn the score function, which is then used to derive the reverse-time SDE.

According to the Fokker-Planck equation, these two processes can be represented by ODEs.

Flow matching: replace conditional forward SDE by ODE

Forward process of diffusion model (DDPM) converges to SDE
[Song et al., 2020]:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dB_t$$

This conditional diffusion path has the form:

$$p_t(x|x') = \mathcal{N}(x|\alpha_t, (1 - \alpha_t^2)I), \quad \alpha_t = e^{-\frac{1}{2}T(t)}, \quad T(t) = \int_0^t \beta(s) ds$$

By Fokker-Planck equation, we derive the conditional flow:

$$dX = -\frac{1}{2}\beta(t) \left(X - \frac{X - \alpha_t}{1 - \alpha_t^2} \right) dt$$

Flow matching: understanding the loss function

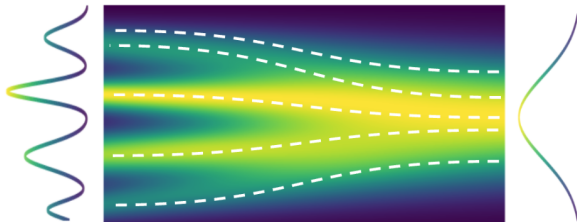


Figure: SDE and reverse-time SDE

DM learns the overall score function $\nabla \log q(x_t)$ by supervising the network using the conditional score function.

According to the Fokker-Planck equation, learning the conditional score function is equivalent to learning the conditional vector field.

Therefore, the following loss function can be viewed as an equivalent loss function to the DM.

$$E_{t,p_t} [\|v_{\theta,t}(x) - u_t(x)\|^2] = E_{t,p_t(x|x_1),p_{\text{empirical}}(x_1)} [\|v_{\theta,t}(x) - u_t(x|x_1)\|^2] + \mathcal{C}$$

Connections between flow matching and diffusion models

- Equivalence:

- The Fokker-Planck equation demonstrates the equivalence of these two models.

- Differences:

- Flow matching learns the vector field, whereas diffusion models learn the score function.
- Flow matching is more straightforward to mathematically define and implement, facilitating its application to various geometries and initial distributions.

Outline

- 1 Brief introduction to SDEs
- 2 From SDEs to diffusion models
- 3 Accelerated sampling
- 4 Flow matching and its connections with other generative models
- 5 Flow matching for learning on manifolds

Flow matching for learning on manifolds

- Brief introduction to geometry
 - Manifold, metric and geodesic
 - Examples
- Flow matching for learning on manifolds

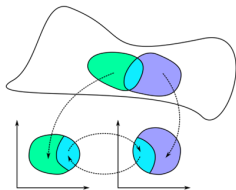
Manifold

$\mathcal{M}$ is a differentiable manifold with dimension m if:

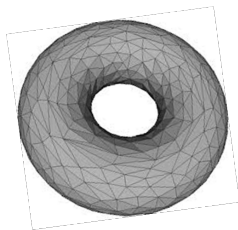
- $\mathcal{M}$ is a Hausdorff and second countable topological space.
- $\mathcal{M}$ has a differentiable atlas $\{\phi_i\}$. Image of ϕ_i is an open set in $\mathbf{R}^m$.

$\mathcal{M}$ is a submanifold of $\mathbf{R}^n$ with dimension m if:

- $\mathcal{M}$ is a subset of $\mathbf{R}^n$.
- $\forall x \in \mathcal{M}, \exists$ openset B s.t. $x \in B, B \cup \mathcal{M}$ diffeomorphic to an open set of $\mathbf{R}^m$.



(a) Atlas



(b) Torus is a submanifold of $\mathbf{R}^3$

Riemannian metric, geodesic

A Riemannian metric g on $\mathcal{M}$ assigns to each $p \in \mathcal{M}$ a positive-definite inner product

$$g_p : T_p\mathcal{M} \times T_p\mathcal{M} \rightarrow \mathbf{R}$$

in a smooth way. A smooth manifold $\mathcal{M}$ endowed with a Riemannian metric g is a Riemannian manifold. Then, the length of a C^1 curve $\gamma : [0, 1] \rightarrow \mathcal{M}$ is defined as:

$$\text{length}(\gamma) = \int_0^1 \sqrt{g(\dot{\gamma}(t), \dot{\gamma}(t))} dt$$

A geodesic is a curve representing in some sense the shortest path between two points in a Riemannian manifold.

Exponential and log mapping

Let $v \in T_p\mathcal{M}$ be a tangent vector to the manifold at p . Then there is a unique geodesic $\gamma : [0, 1] \rightarrow \mathcal{M}$ satisfying $\gamma(0) = p$ with initial tangent vector $\dot{\gamma}(0) = v$. The corresponding exponential map is defined by:

$$\exp_p(v) = \gamma(1)$$

Logarithm mapping is the inverse mapping of $\exp_p$:

$$\exp_p(\log_p(q)) = q$$

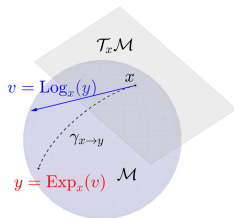


Figure: Exp and log mapping

Laplacian and spectral decomposition

In differential geometry, the Laplace–Beltrami operator is a generalization of the Laplace operator to functions defined on Riemannian manifolds. This operator is a definite operator defined by Riemannian metric g . The spectral decomposition is crucial to understand the manifold:

$$\Delta(\cdot) = \left\langle \sum_i \lambda_i \phi_i(x) \phi_i(y), \cdot \right\rangle$$

where (λ_i, ϕ_i) are eigenvalues and eigenfunctions of Δ .

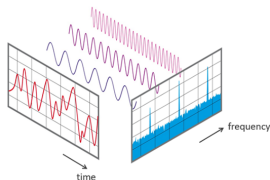


Figure: Example: $\sin(kx)$, $k \in \mathbf{N}$ are eigenfunctions of $\frac{d^2}{dx^2}$ in $[0, 2\pi]$

New metrics derived by spectral decomposition

Although the distance induced by Euclidean is intuitive and useful, it is local dependent, sensitive to topological noise and computational expensive. Therefore people define new distances.

Given the eigenvalues and eigenfunctions (λ_i, ϕ_i) of the Laplacian operator Δ , we can define the following metric:

- Diffusion distance [De Goes et al., 2008]:

$$d(x, y) = \sum_i e^{-\lambda_i t} (\phi_i(x) - \phi_i(y))^2$$

- Commute distance [Fouss et al., 2007]:

$$d(x, y) = \sum_i \frac{1}{\lambda_i} (\phi_i(x) - \phi_i(y))^2$$

- Biharmonic distance [Lipman et al., 2010]:

$$d(x, y) = \sum_i \frac{1}{\lambda_i^2} (\phi_i(x) - \phi_i(y))^2$$

Example: Biharmonic distance and Geodesic distance of Euclidean space



Figure: Geodesic distance vs Biharmonic distance. On general manifolds where the geodesic is not only computationally expensive but can lead to degeneracy (e.g., along boundaries). However, spectral distances (e.g., biharmonic) can be computed efficiently contingent on a one-time processing cost.

Examples: Visualize different metrics

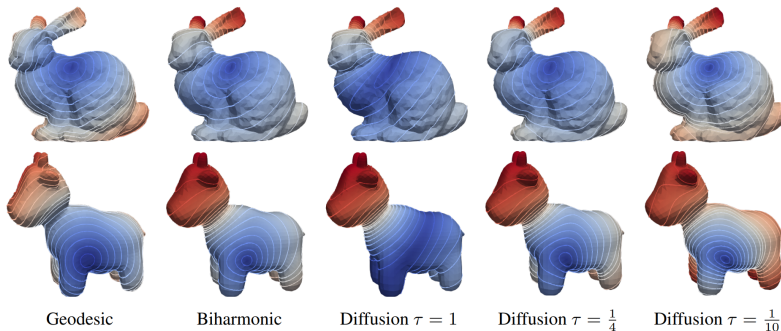


Figure: Contour plots of geodesic and spectral distances (to a source point) on general manifolds. Geodesics are expensive to compute and are globally non-smooth. The biharmonic distance behaves smoothly while the diffusion distance requires careful tuning of the hyperparameter τ .

Riemannian geodesic flow matching

By flow matching, we only need to design conditional flow mapping. The easiest way is to define the conditional flow by:

$$\psi_t(x_0|x_1) = \exp_{x_0}(t \log_{x_0} x_1) = x_t$$

The corresponding conditional flow is given by:

$$v(x_t|x_1) = \dot{x}_t = d(x_0, x_1) \frac{\log_{x_t} x_1}{\|\log_{x_t} x_1\|}$$

The Riemannian conditional flow matching loss function is given by:

$$Loss = E_{t,p,q} \|\nu_\theta(x_t) - \dot{x}_t\|^2$$

where $x_0 \sim p$, $x_1 \sim q$, $x_t = \exp_{x_0}(t \log_{x_0} x_1)$. p and q are the empirical distribution and the noise distribution.

Example: Flow matching on general geometries

[Chen and Lipman, 2024]

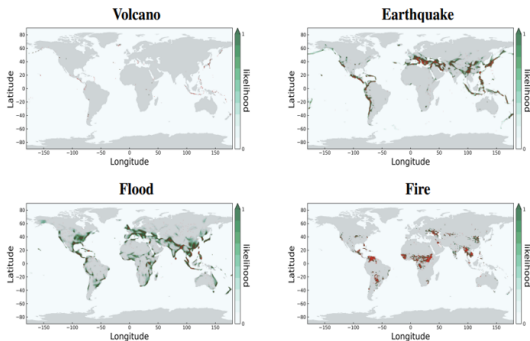


Table 2: Test NLL on Earth and climate science datasets. Standard deviation estimated over 5 runs.

	Volcano	Earthquake	Flood	Fire
Dataset size (train + val + test)	827	6120	4875	12809
<i>CNF-based</i>				
Riemannian CNF (Mathieu & Nickel, 2020)	-6.05 ± 0.61	0.14 ± 0.23	1.11 ± 0.19	-0.80 ± 0.54
Moser Flow (Rozen et al., 2021)	-4.21 ± 0.17	-0.16 ± 0.06	0.57 ± 0.10	-1.28 ± 0.05
CNF Matching (Ben-Hamu et al., 2022)	-2.38 ± 0.17	-0.38 ± 0.01	0.25 ± 0.02	-1.40 ± 0.02
Riemannian Score-Based (De Bortoli et al., 2022)	-4.92 ± 0.25	-0.19 ± 0.07	0.48 ± 0.17	-1.33 ± 0.06
<i>ELBO-based</i>				
Riemannian Diffusion Model (Huang et al., 2022)	-6.61 ± 0.96	-0.40 ± 0.05	0.43 ± 0.07	-1.38 ± 0.05

References I



Albergo, M. S. and Vanden-Eijnden, E. (2022).
Building normalizing flows with stochastic interpolants.
arXiv preprint arXiv:2209.15571.



Chen, R. T. and Lipman, Y. (2024).
Flow matching on general geometries.
In The Twelfth International Conference on Learning Representations.



Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. (2018).
Neural ordinary differential equations.
Advances in neural information processing systems, 31.



De Goes, F., Goldenstein, S., and Velho, L. (2008).
A hierarchical segmentation of articulated bodies.
In Computer graphics forum, volume 27, pages 1349–1356. Wiley Online Library.



Fouss, F., Pirotte, A., Renders, J.-M., and Saerens, M. (2007).
Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation.
IEEE Transactions on knowledge and data engineering, 19(3):355–369.



Ho, J., Jain, A., and Abbeel, P. (2020).
Denoising diffusion probabilistic models.
Advances in neural information processing systems, 33:6840–6851.



Kingma, D., Salimans, T., Poole, B., and Ho, J. (2021).
Variational diffusion models.
Advances in neural information processing systems, 34:21696–21707.

References II



Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. (2022).

Flow matching for generative modeling.

arXiv preprint arXiv:2210.02747.



Lipman, Y., Rustamov, R. M., and Funkhouser, T. A. (2010).

Biharmonic distance.

ACM Transactions on Graphics (TOG), 29(3):1–11.



Liu, X., Gong, C., and Liu, Q. (2022).

Flow straight and fast: Learning to generate and transfer data with rectified flow.

arXiv preprint arXiv:2209.03003.



Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. (2022).

Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps.

Advances in Neural Information Processing Systems, 35:5775–5787.



Luo, C. (2022).

Understanding diffusion models: A unified perspective.



Oksendal, B. (2013).

Stochastic differential equations: an introduction with applications.

Springer Science & Business Media.



Song, Y., Dhariwal, P., Chen, M., and Sutskever, I. (2023).

Consistency models.

arXiv preprint arXiv:2303.01469.

References III



Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2020).
Score-based generative modeling through stochastic differential equations.
arXiv preprint arXiv:2011.13456.



Vincent, P. (2011).
A connection between score matching and denoising autoencoders.
Neural computation, 23(7):1661–1674.